

Yazılım Mimarisi: Seçim, Zamanlama ve Kademeli Değişim

Ömer Faruk Yavuz

August 2026

Özet

Yazılım mimarisi tartışmalarının verimsiz kalmasının başlıca nedeni, tartışmanın hangi katmanda yürüdüğüünün belirlenmemiş olmasıdır; araç seçimi kısıt kararı verilmeden konuşulduğunda, aracın özellikleri kararı geriye doğru belirler. Kararlar değişim maliyetine göre altı katmana ayrılabilir — alan, kısıt, sınır, topoloji, uygulama, işletim — ve bu sıralamada yukarıdaki katman aşağıdakini kısıtlar; ters yönde bir belirlenim gözlemlendiğinde mimari bozulmuş demektir. Veri yapısı ve mimari kalıp seçimlerinin ayrımı da bu çerçeveden çıkar: bir veri yapısını değiştirmek tek bir dosyayı, bağımlılık yönünü değiştirmek her dosyayı etkiler; dolayısıyla ilkinde erken karar zarar, ikincisinde geç karar zarardır. Kalıpların adları — Hexagonal, Clean, CQRS — uygulanacak şeyin kendisi değil, bir fikrin belirli bir uygulama biçimidir; Hexagonal’in altındaki tek fikir, iş mantığının dış bağımlılıklar olmadan sıranabilmesidir ve bu fikir kalıbın klasör yapısı benimsenmeden de uygulanabilir. Bir kod tabanının mimari sağlığı, kalıba benzeyip benzemediğiyle değil üç semptomla ölçülür: iş kuralını test etmek için dış sistem gerekip gerekmediği, küçük bir değişikliğin kaç dosya açtırdığı ve değişiklik sonrası neyin bozulacağına dair belirsizlik. Mevcut ve çalışan bir sistemde mimari iyileştirme, topluca geçiş biçiminde değil, dokunulan yerlerin kademeli olarak sıyrılmasıyla yapılır; çünkü uzun süredir çalışan kod, dokümanite edilmemiş alan bilgisinin fiili deposudur ve topluca yeniden yazım bu bilgiyi siler.

Giriş

Mimari tartışmaları çoğunlukla kalıp adları üzerinden yürütülür: “Hexagonal mi yapalım”, “mikroservise mi geçelim”, “kuyruk mu kullanalım”. Bu biçimdeki bir soru cevaplanamaz, çünkü sorunun içinde hangi problemin çözülmek istendiği yer almaz. Kalıp bir çözümdür; çözüm, problemi belirtmeden değerlendirilemez.

İkinci ve daha sinsi sorun, tartışmaların birbirine karışan katmanlarda yürütülmesidir. Bir ekip “hangi kuyruk hizmeti kullanılсын” sorusunu, “bu işlemin sonucu beklenmek zorunda mı” sorusu cevaplanmadan tartışabilir. Bu durumda tartışma sonuçsuz kalmaz — daha kötüsü olur, yanlış sonuçlanır: seçilen aracın özellikleri, henüz verilmemiş kısıt kararını geriye doğru belirler.

Metin boyunca kullanılan terimlerin tanımları şöyledir. **Mimari**, bir sistemde değiştirilmesi pahalı olan kararların bütünüdür; bu tanım, mimariyi diyagramla ya da klasör yapısıyla tanımlayan yaygın kullanımdan bilinçli olarak ayrılır. **Kalıp** (*pattern*), tekrar eden bir problemin adlandırılmış çözüm biçimidir; adı, çözümün kendisi değil çözüme yapılan bir göndermedir. **Sınır** (*boundary*), bir değişikliğin yayılmasının durduğu yerdir. **Geri alma maliyeti**, verilen bir kararın değiştirilmesi hâlinde kaç dosyanın, kaç sürecin ya da hangi verinin taşınması gerektiğidir.

Yaygın ve yanlış bir kabul, iyi mimarinin bilinen kalıpların uygulanması olduğudur. Bu not bu kabulden ayrılır: kalıplar birer araçtır ve her aracın bir bedeli vardır; bedelin karşılığının doğduğu koşul oluşmadan uygulanan kalıp net zarar üretir.

Notun kapsamı, mimari kararların sınıflandırılması ve bu sınıflandırmanın seçim ile zamanlama üzerindeki sonuçlarıdır. Kapsam dışı olanlar: belirli teknolojilerin karşılaştırılması, performans ölçüm yöntemleri ve dağıtık sistemlerin tutarlılık teorisi.

Veri Yapıları

Veri yapısı seçimi, yapının özelliklerinden değil, üzerinde yapılacak işlemlerin dağılımından türetilir. Doğru soru “hangi yapı iyidir” değil, “bu veriye hangi biçimde, ne sıklıkta erişilecek” sorusudur.

Karşılaştırma

Yapı	Çözdüğü sorun	Bedeli
Dizi (<i>array, slice</i>)	Sıralı erişim, indeksle sabit zamanlı okuma, bellek yerelliği	Ortadan ekleme ve silme, eleman sayısıyla doğrusal
Bağlı liste	Ortadan sabit zamanlı ekleme ve silme, eleman adresinin sabit kalması	Rastgele erişim yok, önbellek davranışı kötü
Hash tablosu	Anahtarla ortalama sabit zamanlı arama	Sıra bilgisi yok, en kötü durumda bozulur, bellek fazlası
Dengeli ağaç, B-ağacı	Sıralı tutma ve aralık sorgusu	Logaritmik erişim, sabit çarpımı yüksek
Yığın (<i>heap</i>)	“En küçük ya da en büyük olanı ver” — öncelik kuyruğu	Yalnızca uç eleman ucuz
Yığıt (<i>stack</i>)	Geri alma, özyineleme, ayrıştırma	—
Kuyruk, çift uçlu kuyruk	Üretici-tüketici ilişkisi, iş sırası	—
Halka tampon	Sabit bellekte akış, son N kayıt	Eski kayıtlar sessizce düşer
Önek ağacı (<i>trie</i>)	Önek araması, otomatik tamamlama	Bellek tüketimi
Küme (<i>set</i>)	Üyelik testi, tekilleştirme	—
Bloom filtresi	“Kesinlikle yok” cevabını çok az bellekle verme	Yanlış pozitif üretir
Ayrık kümeler (<i>union-find</i>)	Bileşen ve eşdeğerlik takibi	Yalnızca birleştirme; ayırma desteklenmez
Komşuluk listesi	Seyrek graf temsili	Yoğun grafta matris daha verimli
LRU önbelleği	Sınırlı bellekte en az kullanılamı atma	İki yapının eşzamanlı tutulması

Tabloda görünmeyen etkenler

Karmaşıklık gösterimi (*big-O*) bir yapının davranışını tam olarak belirlemez, çünkü işlemlerin sabit maliyetlerini gizler. Bağlı liste bunun en bilinen örneğidir: ortadan ekleme işlemi teorik olarak sabit zamanlı olmasına rağmen, modern donanımda çoğu iş yükünde diziden yavaş çalışır. Nedeni, işaretçi takibinin bellekte dağınık adreslere sıçraması ve her sıçramanın önbellek ıskası (*cache*

miss) üretmesidir; dizi ise ardışık bellek kullandığı için tek bir ön bellek satırı okunduğunda birden çok eleman getirilmiş olur. Bağlı listenin gerçek kullanım alanı hız değil, elemanın bellek adresinin sabit kalmasının gerektiği durumlardır: araya girme listeleri, LRU zinciri, işletim sistemi çekirdeği yapıları.

Bloom filtresi ise farklı bir ödünleşmenin örneğidir. Yapı, “bu eleman kümede yok” cevabını kesin olarak, “var” cevabını ise belirli bir hata payıyla verir. Bunun değeri, pahalı bir aramanın çoğu zaman gereksiz olduğu durumlarda ortaya çıkar: filtre “yok” diyorsa disk okuması ya da ağ isteği hiç yapılmaz. Yanlış pozitif bir hata değil, bellekten tasarruf karşılığında kabul edilen bir maliyettir (Bloom, 1970).

Seçim yöntemi

Sıcak yoldaki — yani sık çalışan — her erişim için şu bilgiler yazıldığında yapı kendiliğinden belirlenir: erişim anahtarla mı yoksa sırayla mı yapılıyor, aralık sorgusu var mı, sıra korunmak zorunda mı, boyut sınırlı mı, ekleme ve silme uçlarda mı ortada mı.

Bu sorulardan biri cevaplanamıyorsa yapı henüz seçilemez. Bu durumda en yalın yapı — dizi ya da hash tablosu — kullanılır, sistem ölçülür ve gerekiyorsa değiştirilir. Bu değişikliğin maliyeti düşüktür, çünkü etkisi tek bir modülün içinde kalır.

Mimari Kalıplar

Karşılaştırma

Kalıp	Çözdüğü sorun	Bedeli
Katmanlı mimari	Sorumlulukların ayrılması; en düşük giriş maliyeti	Katmanlar sızar, iş mantığı katmanlara dağılır
Hexagonal / Clean	İş mantığının veritabanı, çerçeve ve arayüzden bağımsızlaşması; bağımlılık yönünün içeri çevrilmesi	Çok sayıda arayüz ve dönüşüm kodu; küçük projede net zarar
MVC	Görünüm ile durumun ayrılması	Denetleyici katmanı şişer
Modüler monolit	Sınırların netleşmesi, dağıtım karmaşıklığı getirmeden	Disiplin gerektirir; derleyici sınırı zorlamaz
Mikroservis	Bağımsız dağıtım, ayrı ölçekleme, ekip özerkliği	Ağ, tutarlılık ve gözlemlenebilirlik maliyeti
Olay güdümlü / yayın-abone	Gönderen ile alanın ayrışması; olayın çok alıcıya yayılması	Akışın izlenmesi zor; sıra ve yineleme sorunları
CQRS	Okuma ve yazma yüklerinin ayrı optimize edilmesi	İki model, senkronizasyon gecikmesi
Olay kaynaklama	Tam geçmiş, denetlenebilirlik, geçmişin yeniden oynatılması	Şema evrimi ve anlık görüntü karmaşıklığı
Boru hattı	Aşamalı dönüşüm; medya ve veri işleme	Geri basınç yönetimi gerekir
Aktör modeli	Paylaşılan durum olmadan eşzamanlılık	Hata ayıklama zorluğu, dene-tim ağacı tasarımı

Kalıbın adı ile altındaki fikrin ayrımı

Kalıplar adlandırıldıklarında bir yan etki üretir: ad, altındaki fikrin yerini alır. Hexagonal Architecture'ın (Cockburn, 2005) altındaki fikir tek cümleyle ifade edilebilir — iş mantığı, dış sistemler olmadan çalıştırılabilir ve sınanabilir olmalıdır. Bağlantı noktaları (*ports*) ve bağdaştırıcılar (*adapters*) bu fikri uygulamanın bir yoludur; tek yolu değildir ve genellikle en pahalı yoludur.

Bu ayrımın pratik sonucu şudur: bir hesaplamayı istek işleyicisinden çıkarıp saf bir fonksiyona almak ya da bir bağımlılığı somut veritabanı tipi yerine arayüz olarak almak, kalıbın adı anılmadan aynı fikrin uygulanmasıdır. Kalıba benzetmek bir amaç değildir; kalıp, elde edilmek istenen özelliğin bir aracıdır.

Aynı ayrım tersi yönde de geçerlidir. Bir kod tabanı bağlantı noktası ve bağdaştırıcı klasörleriyle düzenlenmiş olabilir, ancak iş kuralı hâlâ bir veritabanı kısıtında ya da bir istek işleyicisinde yaşıyorsa fikir uygulanmamıştır. Kalıbın biçimi, sağladığı özelliğin garantisi değildir.

Karar Katmanları

Kararlar, değişim maliyetine göre sıralandığında birbirini kısıtlayan bir düzen ortaya çıkar. Üstteki katman alttakini belirler; ters yönde bir belirlenim gözlemlendiğinde bu, mimarinin bozulduğunun işaretidir.

#	Katman	Ne kararlaştırılır	Yanlışsa sonucu
1	Alan	Kavramlar, değişmezler, kuralların kendisi	Yanlış ürün doğru biçimde inşa edilir
2	Kısıt	Tutarlılık sınırı, senkron/asenkron ayrımı, kimlik şeması, zaman ve birim modeli	Sistem çalışır ancak düzeltilemez hâle gelir
3	Sınır	Modüller, bağımlılık yönü, arayüzler	Her değişiklik çok sayıda dosya açtırır
4	Topoloji	Süreçler, kuyruklar, veri depoları, ağ yerleşimi	Ölçeklenmez; hata izolasyonu sağlanamaz
5	Uygulama	Veri yapıları, algoritmalar, kütüphaneler	Yavaş çalışır
6	İşletim	Dağıtım, gözlemlenebilirlik, ölçekleme eşikleri	Sorun geç fark edilir

Değiştirme maliyetleri sırasıyla şu mertebelerdedir: birinci katmanda ürünün ne olduğunun değişmesi; ikincide veri göçü ve yeniden yazım; üçüncüde aylarla ölçülen yeniden düzenleme; dördüncüde haftalarla ölçülen altyapı işi; beşincide dosya düzeyinde değişiklik; altıncıda yapılandırma değişikliği.

Ara katman: veri modeli

Şema, tabloların şekli, normalleştirme derecesi ve sürümleme kuralı, ikinci ile üçüncü katman arasında sıkışan ve ikisinden de bağımsız evrilmeyen bir küme oluşturur. Aynı sayılmasının pratik gerekçesi şudur: kod bütünüyle yeniden yazılabilir, veri yazılamaz. Çoğu sistemde veri modeli koddan uzun yaşar; bu nedenle veri modeli kararları, kod kararlarından daha yüksek bir katmanda ele alınır.

Kesişen katman: organizasyon

Conway (1968), bir sistemi tasarlayan kuruluşun, kendi iletişim yapısının kopyası olan bir tasarım üretmek zorunda olduğunu gözlemlemiştir. Bu gözlem, organizasyonu gerçek bir mimari katman hâline getirir: ekip sınırları ile modül sınırları uyumadığında, uzun vadede mimari kararlar değil ekip yapısı kazanır. İki ekibin ortak sahip olduğu modül zamanla ikiye ayrılır; tek ekibin sahip olduğu iki servis zamanla birleşir.

Bu ilişkinin tersine çevrilmesi çalışan bir tekniktir: ekip yapısı, arzu edilen modül sınırlarına göre kurulur ve mimarinin o yönde evrilmesi beklenir. Uygulamada ters Conway manevrası olarak adlandırılır (Skelton ve Pais, 2019).

Yön kuralı

Alt katmandaki bir kararın üst katmanı belirlemesi, düzeltilmesi gereken bir durumdur. Dört tipik örnek:

- Alan tipinde serileştirme etiketinin bulunması — dış temsil (4) alana (1) sızmıştır.
- Bir iş kuralının yalnızca veritabanı kısıtında yaşaması — uygulama (5) alanı (1) belirlemiştir.
- “Sunucusuz işlev kullanıyoruz, o hâlde bu işlem asenkron olsun” — topoloji (4) kısıt kararını (2) belirlemiştir.
- Modül sınırının ekip sayısına göre çizilmesi — organizasyon sınırı (3) belirlemiştir.

Bu örneklerin ortak yapısı, aracın ya da mevcut durumun kararın gerekçesi hâline gelmesidir. Kararın gerekçesi problem olmalıdır; araç, karar verildikten sonra seçilir.

Pratik kullanım

Bir mimari tartışmasında ilk yapılacak iş, tartışmanın hangi katmanda yürüdüğünü tespit etmektir. Dördüncü katmandaki bir araç karşılaştırması, ikinci katmandaki kısıt kararı verilmeden yapıldığında sonuçsuz kalmaz — yanlış sonuçlanır, çünkü karşılaştırılan araçların özellikleri, verilmemiş kararın yerine geçer.

Kararların Geri Alınabilirliği

Ayrımın ölçütü

Bir kararın ne zaman verileceğini belirleyen şey önemi değil, geri alınmasının maliyetidir. Ölçüt tek bir soruyla ifade edilebilir: bu karar değiştiğinde kaç yer değişir.

Hash tablosunun ağaç yapısına çevrilmesi tek bir modülü etkiler. İş mantığının nesne-ilişkisel eşleyici (*ORM*) tiplerine bağlanmış olmasının geri alınması ise her dosyayı etkiler. İkisi de teknik karardır, ancak birincisinde erken karar zarar, ikincisinde geç karar zarardır.

Baştan verilmesi gerekenler

- Modül sınırları ve bağımlılık yönü. Hexagonal ve Clean kalıplarının asıl katkısı budur; arayüz sayısı değildir.
- Kimlik şeması: tanımlayıcının ne olduğu, neyi kapsadığı, dışarıya görünüp görünmediği.
- Senkron mu asenkron mu: bir çağrının sonucunun beklenip beklenmediği.
- Tutarlılık modeli: hangi verilerin aynı anda tutarlı olmak zorunda olduğu.
- Veri modelinin şekli.
- Zaman ve birim temsili: depolamanın hangi zaman diliminde tutulduğu, parasal değerlerin hangi tiple ifade edildiği.

Son maddenin bu listede yer almasının nedeni, tek kişilik projelerde bile sonradan düzeltilememesidir. Kayan noktalı sayıyla saklanmış parasal değerler ya da yerel saatle saklanmış zaman damgaları, veri taşınmadan düzeltilemez ve veri taşıma her zaman bilgi kaybı riski taşır.

Sonraya bırakılabilecekler

- Somut veri yapısı seçimleri.
- Önbellek katmanları ve indeksler.
- Tek parçadan servis çıkarma.
- CQRS ve olay kaynaklama.
- Kuyruk, havuz ve eşzamanlılık sınırlarının değerleri.
- Soyutlama çıkarma: ikinci somut uygulama ortaya çıkana kadar beklenir.

Bunların erken kararlaştırılması iki yönlü zarar üretir. Birincisi doğrudan maliyettir: yazılan kod, karşılığı olmayan bir esneklik için ödenmiştir. İkincisi dolaylıdır: erken seçilen soyutlama, ihtiyaç ortaya çıktığında genellikle yanlış yerden geçmiş olur ve yanlış soyutlamayı kaldırmak, hiç soyutlama olmamasından pahalıdır.

Belirsizlik altında karar

Geri alması pahalı olan bir karar, gereken bilgi henüz yokken verilmek zorunda kalınabilir. Bu durumda kullanılabilcek ölçüt şudur: hangi seçenek yanlış çıkarsa daha ucuza dönülür.

Genellikle daha kısıtlı olan seçenek tercih edilir, çünkü sonradan gevşetmek sıkılaştırmaktan kolaydır. Senkron bir işlemi asenkrona çevirmek, tersinden kolaydır. Tanımlayıcıya baştan fazladan bir alan koymak, sonradan eklemekten kolaydır. Verilen bir garantiyi geri çekmek, verilmemiş bir garantiyi sonradan vermekten zordur.

Mevcut Kod Tabanında Uygulama

Kalıp uygulanmamış ve sistem çalışıyorsa

Uzun süredir kalıp uygulamadan çalışan bir sistem, bir gözlemi kanıtlar ve bir başkasını kanıtlamaz.

Kanıtladığı, o ölçekte kalıbın maliyetinin getirisinden büyük olduğudur. Bu doğru bir tespittir ve karşıt yanlış — karşılığı olmayan katman eklemek — en az onun kadar yaygındır.

Kanıtlamadığı ise kalıbın çözdüğü sorunun ortaya çıkmayacağıdır. Bu sorunlar arıza biçiminde gelmez: sistem çökmez, bir istisna fırlatmaz. Belirtisi, bir değişikliğin beklenenden uzun sürmesidir ve bu, çoğunlukla mimariyle ilişkilendirilmez. Hexagonal’in vaadi “kod çalışsın” değil, “üç yıl sonra değiştirmek ucuz olsun” biçimindedir; dolayısıyla çalışıyor olması vaadin sınındığı anlamına gelmez.

Ölçüt olarak semptomlar

Bir kod tabanının mimari sağlığı, kalıp uyumuyla değil üç soruyla ölçülür:

- Bir iş kuralını sınamak için dış bir sistem — veritabanı, kuyruk, üçüncü taraf servisi — ayağa kaldırmak gerekiyor mu?
- Küçük bir değişiklik kaç dosya açtırıyor?
- “Buraya dokunursam ne bozulur” belirsizliği var mı?

Üçü de yoksa yapı o ölçek için yeterlidir. Biri belirmeye başladığında, o belirli soruna karşılık gelen tek düzeltme yapılır; kalıbın bütünü kurulmaz. Semptomu ölçüt olarak kullanmanın avantajı, düzeltmenin ne zaman biteceğini de belirlemesidir: semptom kaybolduğunda iş bitmiştir.

Topluca geçişin başarısızlık nedenleri

Yıllardır çalışan bir sistemde “yeni mimariye geçelim” biçimindeki bir karar iki nedenle başarısız olur.

Birincisi bilgi kaybıdır. Çalışan koddaki dağınıklığın önemli bir kısmı tesadüf değil, karşılaşılacak gerçek durumların izidir: kenar durumlar, tekil müşteri istisnaları, üretimde bulunmuş hatalar. Bunlar çoğunlukla belgelenmemiştir ve yalnızca kodda bulunur. Yeni yapı temiz olur, ancak bu bilgi silindiği için aynı hatalar tek tek yeniden keşfedilir. Büyük yeniden yazımların bilinen başarısızlık örüntüsünün özü budur (Spolsky, 2000).

İkincisi bitiş çizgisinin bulunmayışıdır. Geçiş aylarca sürer, bu süre boyunca yeni işlevsellik akışı yavaşlar ve genellikle tamamlanmadan öncelik değişir. Ortaya çıkan durum, başlangıçtakinden kötüdür: eski yapı, yeni yapı ve ikisi arasındaki köprü aynı anda bakım gerektirir.

Kademeli sıyırma

Çalışan yaklaşım, sistemi yerinde bırakıp kenardan değiştirmektir. Bu yaklaşım literatürde boğan incir kalıbı olarak adlandırılır (Fowler, 2004). Uygulama kuralları şunlardır:

- Yeni yazılan her modül temiz sınırla yazılır. Yeni kod eskiyi çağırabilir; eski kodun içine karışmaz.
- Zaten dokunulmak zorunda kalınan yer düzeltilir. Bir hata için açılan dosyada iş mantığı istek işleyicisinden çıkarılıp saf bir fonksiyona alınır. Bu, ek iş değil, yapılmakta olan işin parçasıdır.
- Sınır önce en çok acıyan yere çekilir: sınanması en zor, en sık değişen ve dokunulmasından en çok çekinilen yer. Böyle bir yer genellikle tektir.
- Dokunulmayan kod dokunulmadan bırakılır. Yıllardır değişmemiş ve çalışan bir modülü tutarlılık gerekçesiyle taşımak net zarardır: risk alınır, karşılığında hiçbir şey kazanılmaz.

Bu yaklaşımın başarı ölçütü de kalıp uyumu değildir. Bir yıl sonra yukarıdaki üç semptom sorusunun cevabı iyileşmişse yaklaşım işe yaramıştır; kod tabanının kalıba benzeyip benzemediği ölçüt değildir.

Mimari borcun niteliği

Mimari borç kavramı, ödenmemesi gereken bir yükümlülük olarak anlaşıldığında yanlış kullanılır. Metafor ilk formüle edildiğinde kastedilen, teslimatı hızlandırmak için bilinçli olarak alınan ve faiziyle geri ödenen bir borçtu (Cunningham, 1992). Çalışan bir ürün üretmiş bir sistemde biriken mimari borç, ürünün var olmasının bedeli olarak ödenmiş bir borçtur.

Ayrım şuradadır: borcun varlığı sorun değildir, farkında olunmaması ve faizinin ölçülmemesi sorundur. Faiz burada somut olarak ölçülebilir — bir değişikliğin ne kadar sürdüğü ve kaç dosya açtırdığı.

Sonuç

Kalıp adları üzerinden yürütülen mimari tartışmaları, tartışılan şeyin ne olduğunu gizler. Kalıbın adı bir çözüme yapılan göndermedir; gönderme, çözülmek istenen problem belirtilmeden değerlendirilemez. Bu notta kurulan çerçevenin pratik karşılığı, her mimari sorunun önce iki soruya indirgenmesidir: bu karar hangi katmanda veriliyor ve değiştirilmesi kaç yeri etkiliyor.

Bu iki soru, seçim ile zamanlamayı birbirinden ayırır. Veri yapısı seçimi ile bağımlılık yönü kararı aynı türden kararlar değildir; birincisi ölçülerek verilir, ikincisi tahmin edilerek. Erken optimizasyonun zarar olması ile erken sınır çizmenin kazanç olması arasında çelişki yoktur — ikisi farklı katmanların kararlarıdır ve maliyet profilleri terstir.

Çerçevenin sınırı, katman ayrımının her zaman net olmayışıdır. Veri modeli kararları ikinci ile üçüncü katman arasında sıkışır; organizasyon bütün katmanları keser. Ayrıca kararların geri alınabilirliği bir tahmindir: bir kararın ne kadar yayılacağı ancak sistem büyüdükçe kesinleşir ve bazı kararlar öngörülenden ucuz, bazıları öngörülenden pahalı çıkar.

Açık kalan soru, semptom temelli ölçütün ne kadar erken kullanılabileceğidir. “Bir değişiklik kaç dosya açtıyor” sorusu, yeterince değişiklik yapılmış bir sistemde anlamlıdır; sıfırdan başlayan bir projede ölçülecek bir geçmiş yoktur. Bu boşlukta karar, ölçüme değil benzer sistemlerdeki deneyime dayanır — ve bu, çerçevenin ölçülebilirlik iddiasının en zayıf olduğu noktadır.

Kalıpların bütünüyle reddedilmesi de bu notun sonucu değildir. Kalıplar, çözdükleri problem ortaya çıktığında hazır ve sınanmış çözümlerdir; sağladıkları asıl fayda, o problemi yeniden keşfetmek zorunda kalmamaktır. Sorun kalıpların kendisi değil, problem oluşmadan uygulanmalarıdır.

Kaynaklar

- Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7).
- Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)*.
- Conway, M. E. (1968). How Do Committees Invent? *Datamation*, 14(4).
- Cunningham, W. (1992). The WyCash Portfolio Management System. *OOPSLA '92 Experience Report*.
- Fowler, M. (2004). *StranglerFigApplication*. martinowler.com
- Fowler, M. (2011). *CQRS*. martinowler.com
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.
- Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12).

- Skelton, M. ve Pais, M. (2019). *Team Topologies*. IT Revolution.
- Spolsky, J. (2000). Things You Should Never Do, Part I. *Joel on Software*.